9 Lab 9: Use of GPU memory

Objectives

- Explore the utilization of global memory, constant memory, and shared memory in parallel algorithms.
- Understand the impact of different memory types on performance and identify optimization opportunities.
- Familiarize oneself with CUDA kernel design and execution for parallel processing on NVIDIA GPUs.

Deliverables

- Complete and functional CUDA C++ code for vector addition with distinct kernels utilizing global, constant, and shared memory.
- Documentation detailing the rationale and methodology behind memory type selection for each kernel.
- Performance analysis report comparing the execution times and efficiency of the different memory implementations.

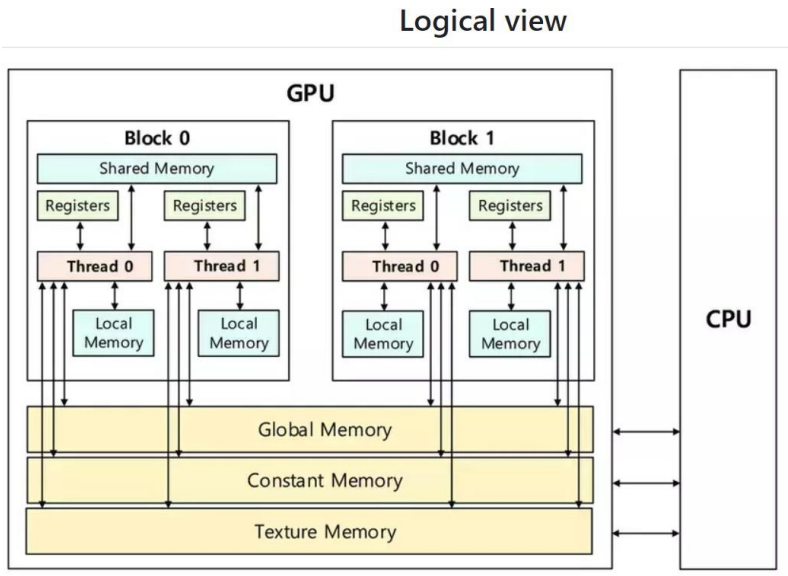
9.1 GPU memory

Before delving into the various memory types within a GPU, it's important to understand that when we talk about memory, we generally categorize it into two main types: physical memory and logical memory.

Physical Memory: This refers to the actual hardware memory in a computer. It includes components such as RAM modules and storage devices like hard drives (HDD/SSD). Physical memory is where data and programs are stored directly and can be accessed quickly by the processor.

Logical Memory (Virtual Memory): This is the address space that the operating system and programs can access. Logical memory doesn't necessarily have a direct one-to-one correspondence with physical memory. The operating system typically manages the mapping between logical addresses and physical addresses. This management helps allocate and manage memory for programs running on the system.

You can understand it in a simple way: when we code, we interact with logical memory, and once the code is finished, the data located in logical memory will be mapped to physical memory (meaning the computer will operate in physical memory). Now that we have a foundational understanding of memory, let's explore the specific memory types within a GPU and their purposes.



As mentioned, Blocks and Threads are logical concepts, and due to the SIMT mechanism, it's important to understand how Threads and Blocks are distributed and managed within the logical memory of the GPU. Here, we have a familiar concept known as scope, which plays a crucial role in understanding how resources like Threads and Blocks are allocated and managed within the logical memory of the GPU.

- **Local Memory:** Each Thread can use its own local memory, where it can store temporary variables. This has the smallest scope and is dedicated to each individual Thread.
- **Shared Memory:** Threads within the same Block can share data through shared memory. This allows Threads within the same Block to communicate and access data faster compared to accessing global memory.
- **Global Memory:** Global memory in GPU programming is a fundamental storage space accessible by all threads in all blocks. It serves as the primary repository for data that needs to be shared across threads. However, it's crucial to be mindful of its relatively slower access speed compared to other memory types. To optimize performance, developers should minimize global memory accesses and adopt coalesced memory access patterns. Coalescing, which involves arranging memory accesses in a way that maximizes throughput, can significantly enhance the efficiency of global memory usage.
- **Texture Memory and Constant Memory:** Constant memory provides a read-only, cached space that is shared among all threads within a block. This type of memory is ideal for storing values that remain constant across multiple threads, offering fast and efficient access. To harness the benefits of constant memory, developers should focus on using it for read-only data and ensuring that access patterns are aligned and coalesced. It's essential to be mindful of the limited capacity of the constant cache and avoid exceeding it to maintain optimal performance.

- **Shared memory** is a high-speed, block-specific memory that is shared among threads within the same block. It is explicitly managed by the programmer and serves as a space for creating local copies of data that multiple threads need to access and modify concurrently. To maximize the advantages of shared memory, developers should efficiently use it to reduce reliance on global memory accesses. Strategies include minimizing bank conflicts, which occur when multiple threads access the same memory bank simultaneously, and exploiting shared memory for data reuse and inter-thread communication.

9.2 Code Example:

This CUDA C++ code implements a vector addition program, showcasing three different kernels for parallel execution on a GPU. The first kernel ('vectorAddGlobal') utilizes global memory for vector addition, with each thread handling an element of the vectors. The second kernel ('vectorAddConstant') demonstrates the use of constant memory for one of the vectors, enhancing read-only data access. The third kernel ('vectorAddShared') employs shared memory to cache portions of both vectors, reducing global memory accesses and promoting inter-thread communication within a block. The example initializes host vectors, allocates and copies data to the GPU device, performs vector addition using each kernel, and finally retrieves the results back to the host for verification. While this example may not exhibit significant performance gains due to the simplicity of vector addition, the principles illustrated become crucial for optimizing more complex GPU algorithms with intricate memory access patterns.

```
#include <iostream>
#include <cmath>
#include <cuda_runtime.h>

const int N = 1024; // Size of the vectors

// Kernel for vector addition using global memory
__global__ void vectorAddGlobal(float* a, float* b, float* c)
    ↪ {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        c[i] = a[i] + b[i];
    }
}

// Kernel for vector addition using constant memory
__constant__ float constVector[N]; // Constant memory
    ↪ declaration

__global__ void vectorAddConstant(float* a, float* c) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
```

```
        c[i] = a[i] + constVector[i];
    }
}

// Kernel for vector addition using shared memory
__global__ void vectorAddShared(float* a, float* b, float* c)
    ↪ {
    __shared__ float sharedA[N];
    __shared__ float sharedB[N];

    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        sharedA[i] = a[i];
        sharedB[i] = b[i];
        __syncthreads(); // Ensure all threads have finished
            ↪ copying

        c[i] = sharedA[i] + sharedB[i];
    }
}

int main() {
    // Host vectors
    float h_a[N], h_b[N], h_c[N];

    // Initialize host vectors
    for (int i = 0; i < N; ++i) {
        h_a[i] = i;
        h_b[i] = i * 2;
    }

    // Allocate device memory
    float *d_a, *d_b, *d_c;
    cudaMalloc((void**)&d_a, N * sizeof(float));
    cudaMalloc((void**)&d_b, N * sizeof(float));
    cudaMalloc((void**)&d_c, N * sizeof(float));

    // Copy host data to device
    cudaMemcpy(d_a, h_a, N * sizeof(float),
        ↪ cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, N * sizeof(float),
        ↪ cudaMemcpyHostToDevice);

    // Global memory version
    vectorAddGlobal<<<(N + 255) / 256, 256>>>(d_a, d_b, d_c);
```

```
    // Copy result back to host
    cudaMemcpy(h_c, d_c, N * sizeof(float),
        ↪ cudaMemcpyDeviceToHost);

    // Check results...

    // Reset device memory for the next kernel
    cudaMemset(d_c, 0, N * sizeof(float));

    // Constant memory version
    cudaMemcpyToSymbol(constVector, h_b, N * sizeof(float));
        ↪ // Copy data to constant memory
    vectorAddConstant<<<(N + 255) / 256, 256>>>(d_a, d_c);

    // Copy result back to host
    cudaMemcpy(h_c, d_c, N * sizeof(float),
        ↪ cudaMemcpyDeviceToHost);

    // Reset device memory for the next kernel
    cudaMemset(d_c, 0, N * sizeof(float));

    // Shared memory version
    vectorAddShared<<<(N + 255) / 256, 256>>>(d_a, d_b, d_c);

    // Copy result back to host
    cudaMemcpy(h_c, d_c, N * sizeof(float),
        ↪ cudaMemcpyDeviceToHost);

    // Free device memory
    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);

    return 0;
}
```

Lab 9: Use of GPU Memory

1. **Task 1:** Write a CUDA program to perform vector addition using global memory and compare its performance with a version using shared memory.

2. **Task 2:** Implement a CUDA program that uses constant memory to store a fixed array and performs operations on it.